

SQL Essential Commands Guide

Comprehensive Reference for UPDATE, DELETE, ALTER, TRUNCATE, and DROP

Reference Schema: employees Table

All examples in this document refer to the following standard table structure:

emp_id (INT)	first_name (VARCHAR)	department (VARCHAR)	salary (DECIMAL)	status (VARCHAR)
101	Alice Vance	Engineering	85000.00	Active
102	Bob Smith	Sales	62000.00	Active
103	Charlie Brown	Marketing	54000.00	On Leave

1. UPDATE

DML COMMAND

Modifies existing records in a table based on specified conditions.

SYNTAX

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

PRACTICAL EXAMPLES

```
-- Example A: Give a 10% raise to all Engineering employees
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'Engineering';

-- Example B: Update multiple columns for a specific employee
UPDATE employees
SET department = 'Executive', salary = 95000.00
WHERE emp_id = 101;
```

Crucial Notes & Best Practices:

- **Always use WHERE:** Omitting the WHERE clause updates *every single row* in the table.
- **Transactional Safety:** Execute inside a transaction (BEGIN TRANSACTION; ... ROLLBACK;) to verify modified row count before committing changes.
- **Foreign Keys:** Updating primary key columns may trigger CASCADE updates if foreign key constraints exist.

2. DELETE

DML COMMAND

Removes existing rows from a table based on a condition. Row-by-row deletion that logs each operation.

SYNTAX

```
DELETE FROM table_name
WHERE condition;
```

PRACTICAL EXAMPLES

```
-- Example A: Delete a specific employee by ID
DELETE FROM employees
WHERE emp_id = 103;

-- Example B: Delete all employees with salary below a threshold
DELETE FROM employees
WHERE salary < 30000.00;
```

Crucial Notes & Best Practices:

- **Rollback Possible:** Because DELETE is a DML statement, rows deleted within a transaction can be recovered using ROLLBACK.
- **Performance:** Slower on huge datasets compared to TRUNCATE because each deleted row is recorded individually in transaction logs.
- **Space Reclamation:** Does not immediately free disk space or reset AUTO_INCREMENT values in most RDBMS.

3. ALTER

DDL COMMAND

Modifies the structure of an existing database object (e.g., adding, modifying, or dropping columns/constraints).

COMMON SYNTAXES & EXAMPLES

```
-- Add a new column
ALTER TABLE employees
ADD email VARCHAR(255);

-- Drop an existing column
ALTER TABLE employees
DROP COLUMN status;

-- Modify data type or constraints (PostgreSQL / MySQL variation)
ALTER TABLE employees
MODIFY COLUMN salary DECIMAL(12,2); -- MySQL

-- Add a Constraint
ALTER TABLE employees
ADD CONSTRAINT chk_salary CHECK (salary > 0);
```

Crucial Notes & Best Practices:

- **Table Lock:** Altering large tables can lock the table for extended periods, impacting production availability.
- **Data Truncation:** Shrinking column sizes (e.g., VARCHAR(255) to VARCHAR(50)) will fail if existing data exceeds the new size limit.
- **Syntax Differences:** Exact keywords vary slightly between SQL dialects (Oracle uses MODIFY, PostgreSQL uses ALTER COLUMN ... TYPE).

4. TRUNCATE

DDL COMMAND

Removes all records from a table quickly by deallocating the data pages. Keeps table structure intact.

SYNTAX

```
TRUNCATE TABLE table_name;
```

PRACTICAL EXAMPLE

```
-- Wipe all employee data while preserving table structure & indexes
TRUNCATE TABLE employees;
```

Crucial Notes & Best Practices:

- **High Speed:** Minimally logged operation; vastly faster than DELETE FROM table on millions of rows.
- **Resets Identity:** Resets AUTO_INCREMENT / IDENTITY counters back to initial seed values.
- **No Triggers:** Does NOT fire row-level DELETE triggers.
- **Foreign Key Restriction:** Cannot be executed on tables referenced by active Foreign Key constraints (must disable FKs first).

5. DROP

DDL COMMAND

Completely removes a table, view, index, or database from the database system, destroying both data and schema.

SYNTAX

```
DROP TABLE table_name;  
DROP TABLE IF EXISTS table_name; -- Prevents error if table doesn't exist
```

PRACTICAL EXAMPLES

```
-- Safely drop the employees table  
DROP TABLE IF EXISTS employees;  
  
-- Drop an entire database schema  
DROP DATABASE company_db;
```

Crucial Notes & Best Practices:

- **Irreversible:** Completely deletes table structure, rows, indexes, constraints, and permissions. Cannot be undone via simple transaction rollback in most engines.
- **Cascade Option:** Use `DROP TABLE employees CASCADE;` in engines like PostgreSQL to automatically drop dependent objects (e.g., views, foreign key references).

Command Comparison Matrix

Command	Category	Target Scope	Speed / Overhead	Rollbackable?	Resets Identity?
UPDATE	DML	Specific Data Rows	Moderate (Row-level logging)	Yes (in Transaction)	No
DELETE	DML	Specific/All Rows	Slow (Logs each row)	Yes (in Transaction)	No
ALTER	DDL	Table Structure	Varies (Locks table)	No (Implicit Commit*)	N/A
TRUNCATE	DDL	All Table Rows	Extremely Fast (Minimal log)	No (In most RDBMS)	Yes
DROP	DDL	Whole Object & Structure	Instant	No	N/A (Destroys table)

* Note: PostgreSQL supports transactional DDL allowing ALTER/TRUNCATE rollback, whereas MySQL and Oracle auto-commit DDL statements immediately.